MAE 204

Final Project Report

Pick & Place Cube CoppeliaSim

Brandon Lopez: b1lopez@ucsd.edu

Github Repo

The project implements a complete simulation of a mobile manipulation pick-and-place task using a youBot model (Scene 6). The main code combines 3 core functions: trajectory planning, feedback control, and next state configuration which allows the mobile manipulator to pick up a cube from an initial location and move it to a desired final configuration. The program and functions used were written in MATLAB and are organized into modular functions that correspond to the main components of the mobile manipulation pipeline. The core functions also make use of several functions from the Modern Robotics software library available on the Modern Robotics GitHub repository.

The first component is the **NextState function**, which computes the next configuration of the robot using first-order Euler integration. The robot state includes the chassis pose, arm joint angles, and wheel angles. Joint and wheel velocities are first limited by a maximum velocity constraint to prevent unrealistic commands. The arm joints and wheel angles are updated directly using Euler integration. Wheel motions are then converted into a chassis body twist using the youBot wheel kinematics matrix. The body twist is integrated to update the chassis pose in the world frame, allowing the mobile base to move consistently with the commanded wheel speeds.

The second component is the **TrajectoryGenerator function**, which produces a reference trajectory for the end-effector. The trajectory describes the full pick-and-place sequence including approaching the cube, grasping it, transporting it to the final position, and releasing it. The trajectory is expressed as a sequence of end-effector configurations represented by rotation matrices and position vectors along with the state of the gripper. The trajectory is generated using the ScrewTrajectory function from the Modern Robotics library with quintic time scaling in order to ensure smooth motion between configurations. The trajectory is divided into eight sequential phases:

1. Moving to a standoff configuration above the cube,
2. Descending to the grasp configuration
3. Closing the gripper
4. Returning to the standoff position
5. Moving to the final cube location
6. Descending again
7. Opening the gripper
8. Returning to a standoff configuration.

The resulting trajectory is stored as a matrix where each row represents a desired end-effector configuration at a discrete timestep.

The third component is the **FeedbackControl function**, which computes the commanded end-effector twist required to track the reference trajectory. The controller follows the feedforward-plus-feedback control law described in the course materials. First, a feedforward twist is computed using the transformation between the current desired configuration and the next desired configuration along the reference trajectory. Next, the current configuration error between the actual end-effector pose and the desired pose is computed using the matrix logarithm representation of SE(3). These calculations rely on Modern Robotics library functions

such as MatrixLog6, TransInv, se3ToVec, and Adjoint. The controller then combines the feedforward twist with proportional and integral feedback terms. This produces a commanded body twist that drives the end-effector toward the desired configuration while reducing tracking error over time.

The main wrapper script integrates these three components into a complete simulation. At each timestep the script computes the current end-effector configuration using forward kinematics, obtains the desired configuration from the reference trajectory, and computes the commanded twist using the feedback controller. The forward kinematics and Jacobian calculations rely on functions from the Modern Robotics library such as FKinBody and JacobianBody. The commanded twist is then converted into joint and wheel velocities using the robot Jacobian and a damped least-squares pseudoinverse. The damping factor λ is used to reduce the effects of singularities and improve numerical stability when the Jacobian approaches rank deficiency. This approach provides a simple form of singularity robustness, which helps prevent excessively large joint velocities when the manipulator approaches configurations with poor manipulability.

During the simulation, the program also evaluates manipulability factors for both the angular and linear components of the Jacobian. These values are computed using the eigenvalues of the matrices: $A_w = J_w J_w^T$ and $A_v = J_v J_v^T$. Plotting these values provides insight into how close the manipulator is to singular configurations during the task.

One interesting observation when implementing the complete code was that the trajectory tracking accuracy depends strongly on the controller gains and the damping parameter used in the Jacobian inversion. Without damping, the pseudoinverse solution can produce unstable or excessively large joint velocities when the Jacobian becomes ill-conditioned. Introducing the damped least-squares solution improves numerical stability and produces smoother motion.

Overall, the implemented software successfully simulates the complete mobile manipulation task. The robot follows the reference trajectory, grasps the cube, transports it to the final location, and releases it while maintaining a reasonable tracking error of less than 12%.

Case 1: "Best Controller"

The best-case controller was implemented using a feedforward-plus-PI controller with gains:

$$K_p = 3I_{6x6} \text{ and } K_i = 0.001I_{6x6}$$

With a timestep of $0.01s$. The damped least-squares inverse kinematics solution had a damping factor of $\lambda = 0.01$, in order to make the algorithm robust near singularities. The cube initial and final configurations for this case were the following:

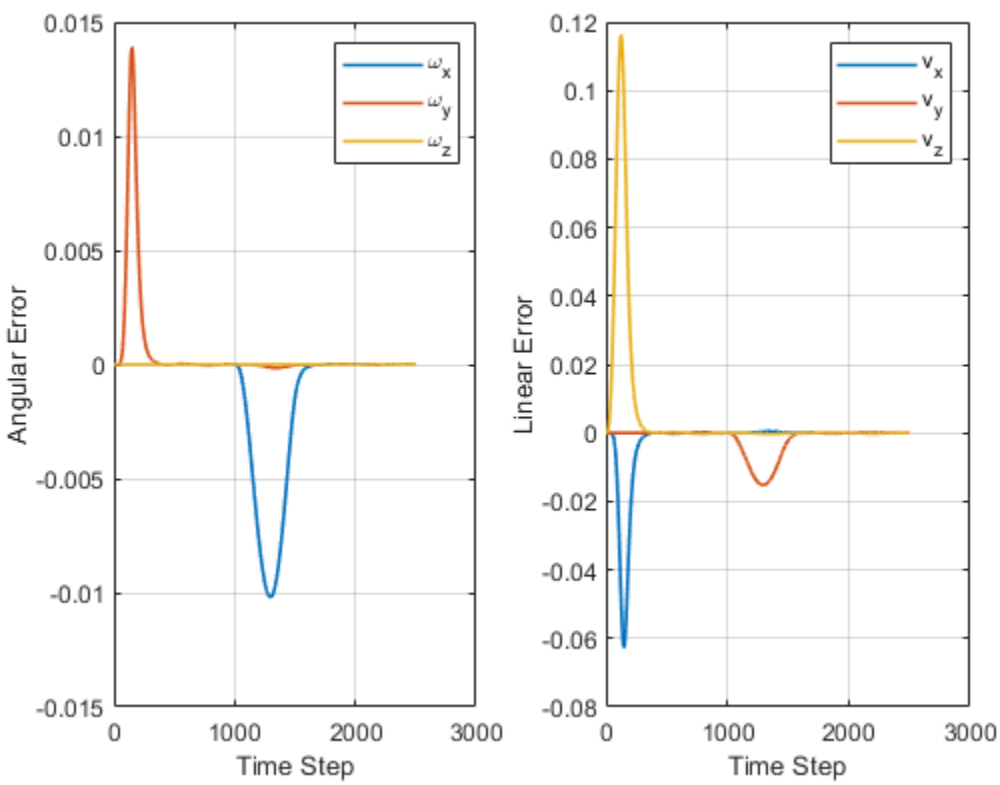
$$(x_{init}, y_{init}, z_{init}, \phi_{init}) = (1, 0, 0.025, 0)$$

$$(x_{final}, y_{final}, z_{init}, \phi_{final}) = (0, -1, 0.025, -\tfrac{\pi}{2})$$

Video Link: [Best Controller]

This was the "best" controller case because the proportional gain is moderate and the integral gain is very small, which leads to smooth convergence towards zero. The feedforward term drives the end-effector along the reference trajectory, while the proportional and integral terms gradually reduce the tracking error. Increasing the integral gain too much leads to aggressive corrective motion, meaning, near singularies there will be an aggressive correction that could result in unstable solutions.

The plot of the six components of X_{err} shows that the controller tracks the reference trajectory well and that the errors remain small over the entire simulation. In the angular error plot, the largest positive peak appears in ω_y which goes up to around 1.5% (Figure 1a). In the linear error plot, the largest positive peak appears in v_z around 11.2% (Figure 1b). After these initial transients, both the linear and angular error quickly decrease and remain close to zero for most of the trajectory and do not exhibit repeated oscillation. This behavior is consistent with the expected case of a well-tuned controller.



a) **b)**

Figure 1: End-effector tracking error X_{err} for the best case controller. a) angular error components as a function of time step and b) linear error components as a function of time step.

Overall, the key takeaway from these plots is that the transient response is visible, but it is still smooth and damped. The error components do not oscillate over long periods of time and remain close to zero. The response is not unrealistically fast, but it is stable, smooth, and free of significant overshoot.

The manipulability plot provides additional information about how the robot moves through the task. The angular manipulability factor, $\mu_1(A_w)$, rises quickly to a peak near 0.33 early in the motion and then settles to an approximately constant value near 0.10 for most of the simulation. This suggests the robot moves away from a less favorable initial angular configuration and then remains in a relatively stable angular region. The linear manipulability factor, $\mu_1(A_v)$, stays lower overall, varying approximately between 0.01 and 0.045 (Figure 2). It shows a small early peak, then decreases, and later rises again toward the end of the motion. This indicates that the translational movement of the robot changes more throughout the task than the angular movement, which is reasonable because the arm and base must coordinate differently during approach, transport, and placement.

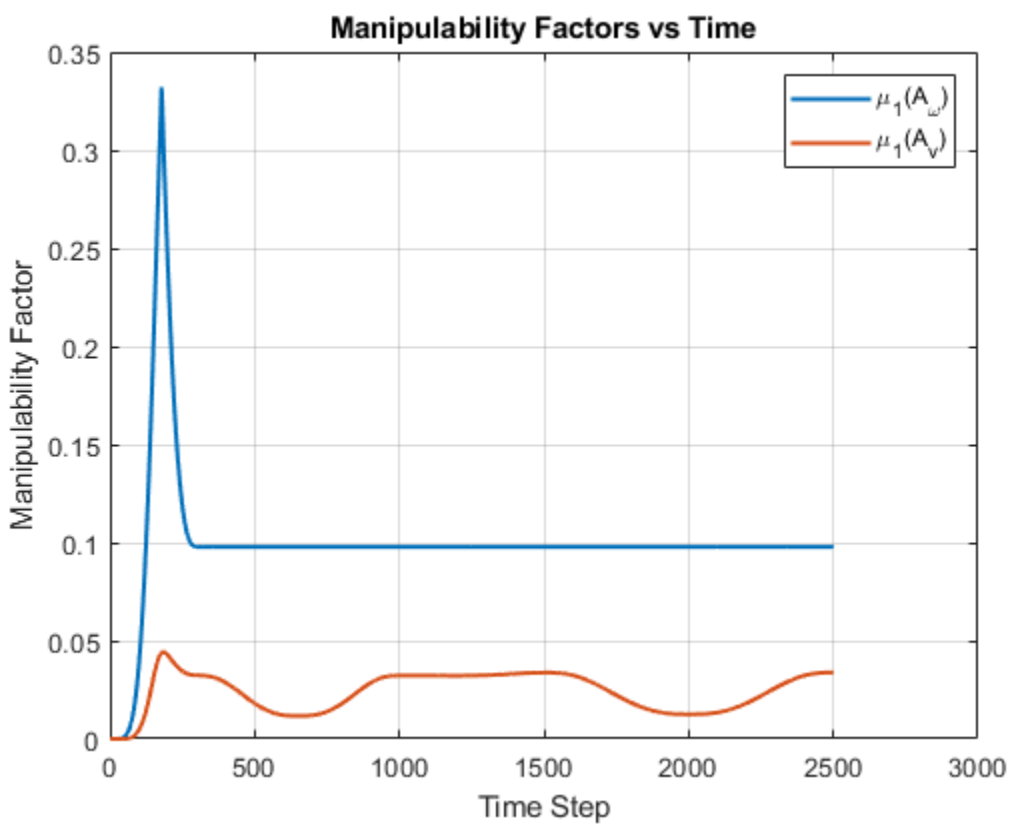


Figure 2: Manipulability factors $\mu_1(A_w)$ and $\mu_1(A_v)$ versus time for the best-case controller. The angular and linear manipulability measures are plotted as the robot executes the pick-and-place trajectory.

Overall, the best-case results show that the selected controller gains a smooth and reliable pick-and-place motion. The tracking errors remain small, converge toward zero, and do not display prolonged oscillation. The manipulability plots also confirm that although factors vary during the task, especially for the linear velocity component, the controller and damped inverse kinematics method are robust enough to maintain stable motion throughout the simulation.

Case 2: "Overshoot Controller"

The overshoot controller was implemented using a feedforward-plus-PI controller with gains:

$$K_p = 15 \text{ and } K_i = 2I_{6x6}$$

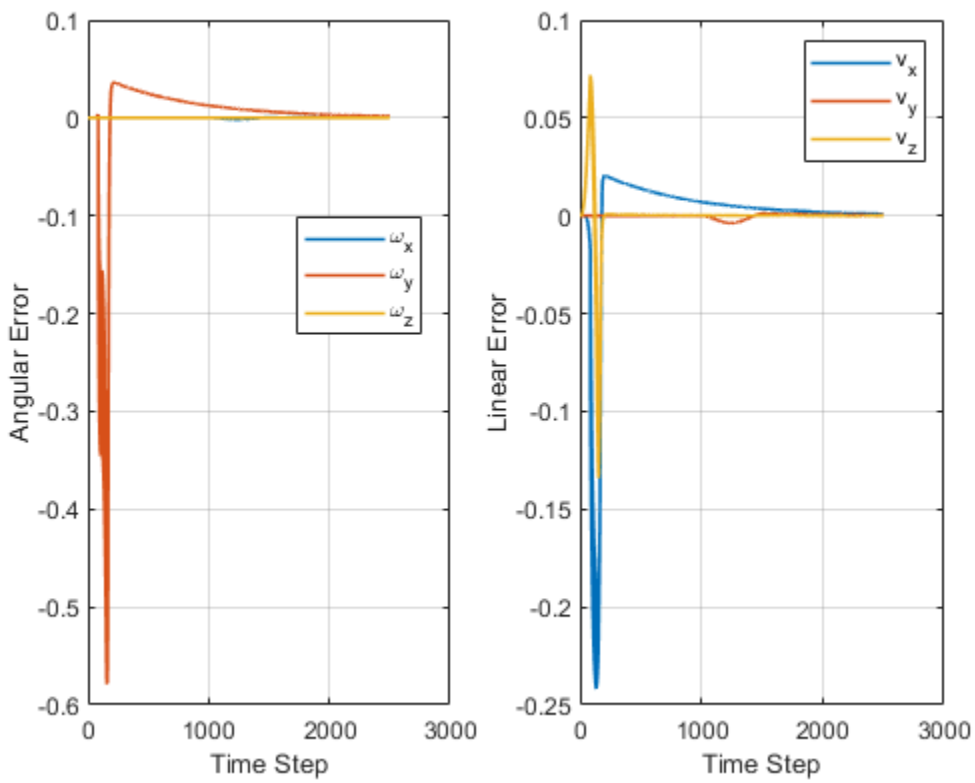
With a timestep of $0.01s$. The damped least-squares inverse kinematics solution had a damping factor of $\lambda = 0.01$, in order to make the algorithm robust near singularities. The cube initial and final configurations for this case were the following:

$$(x_{init}, y_{init}, z_{init}, \phi_{init}) = (1, 0, 0.025, 0)$$

$$(x_{final}, y_{final}, z_{init}, \phi_{final}) = (0, -1, 0.025, -\frac{\pi}{2})$$

Video Link: [Overshoot Controller]

The plot of the six components of X_{err} exhibits noticeable overshoot during the initial portion of the motion. In the angular error plot, the largest negative peak appears in ω_y which goes up to around 57% (Figure 3a). In the linear error plot, the largest positive peak appears in v_x around 23% (Figure 3b). After these initial transients, both the linear and angular error quickly decrease and remain close to zero for most of the trajectory and do not exhibit repeated oscillation. Nevertheless, the configuration of the robot violates joint angle limits and comes up with an unreasonable solution.



a) **b)**

Figure 3: End-effector tracking error X_{err} for the overshoot case controller. a) angular error components as a function of time step and b) linear error components as a function of time step.

The manipulability plot in Figure 4 shows the variation of the angular and linear manipulability factors as the robot executes the pick-and-place motion. The angular manipulability $\mu_1(A_w)$ quickly rises to a peak before stabilizing near a constant value around 0.1 for most of the trajectory. The linear manipulability $\mu_1(A_v)$ varies more throughout the task, reflecting the changing arm and base configurations required to follow the trajectory. Despite the more aggressive controller gains and the resulting overshoot behavior, the damped least-squares inverse kinematics method maintains numerical stability and allows the robot to successfully complete the task.

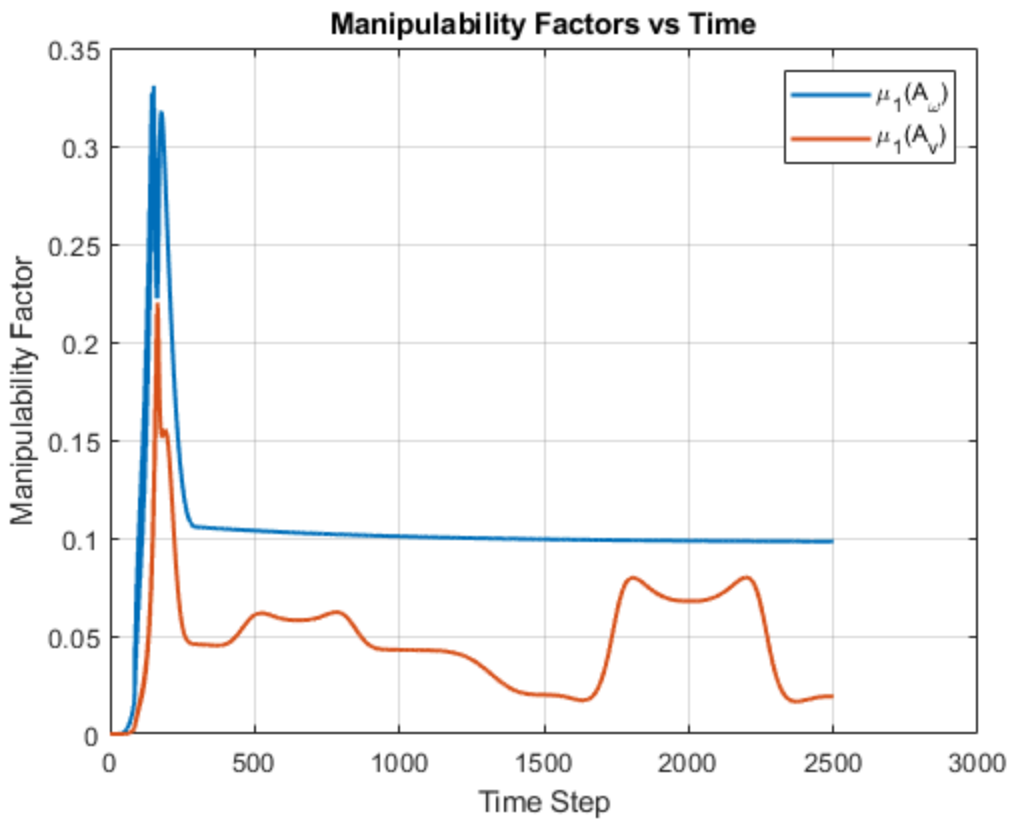


Figure 4: Manipulability factors $\mu_1(A_w)$ and $\mu_1(A_v)$ versus time for the overshoot case controller. The angular and linear manipulability measures are plotted as the robot executes the pick-and-place trajectory.

Even though the controller produced a large initial overshoot, the robot was still able to successfully grasp the cube and place it at the correct final location. However, the aggressive controller gains caused large transient motions and accumulated tracking error early in the trajectory. As a result, some arm configurations became physically unrealistic in the simulation, with links appearing to intersect with each other and even pass through the robot chassis. While the simulation allows these configurations because no collision constraints are enforced, such behavior would not be physically possible in a real robotic system. This observation highlights the importance of proper controller tuning and the potential need for additional constraints, such as collision avoidance, when implementing a controller on a physical robot.

Case 3: "New-Configuration Controller"

The new-configuration controller was implemented using a feedforward-plus-PI controller with gains:

$$K_p = 3I_{6x6} \text{ and } K_i = 0.001I_{6x6}$$

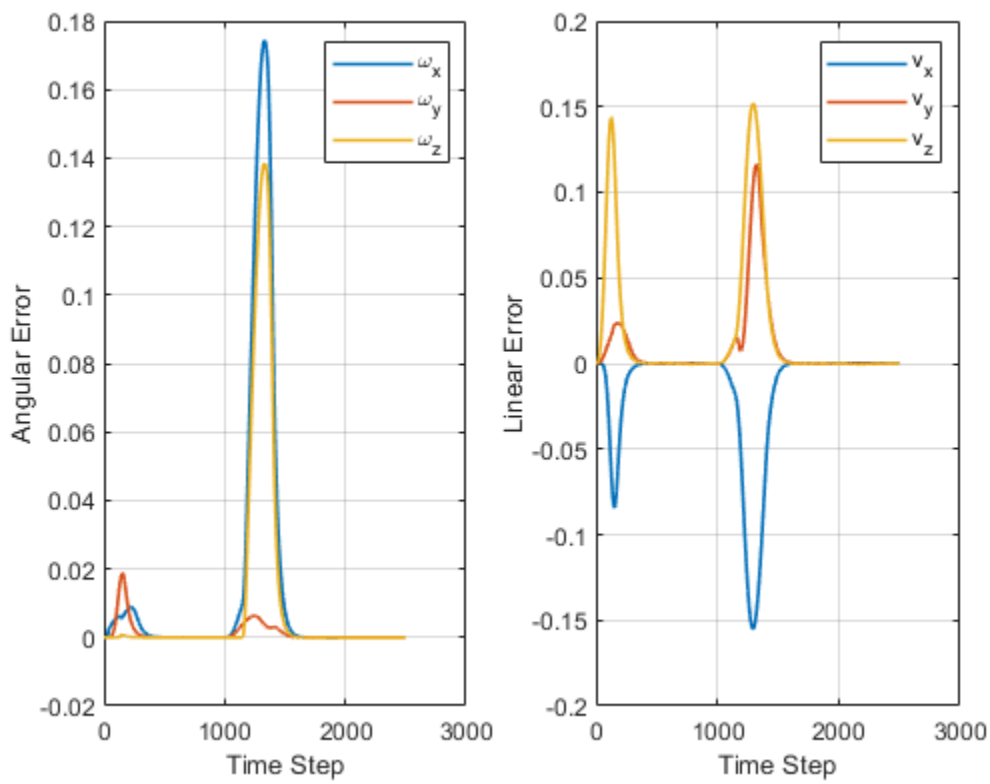
With a timestep of $0.01s$. The damped least-squares inverse kinematics solution had a damping factor of $\lambda = 0.01$, in order to make the algorithm robust near singularities. The cube initial and final configurations for this case were the following:

$$(x_{init}, y_{init}, z_{init}, \phi_{init}) = (1.1, 0.8, 0.025, 0)$$

$$(x_{final}, y_{final}, z_{init}, \phi_{final}) = (1.0, -0.7, 0.025, -\pi)$$

Video Link: [New-Case Controller]

As shown in Figure 5, the angular and linear error plots contain two distinct spikes during the trajectory. These spikes occur when the manipulator passes through configurations that are close to singularities. Near singular configurations the Jacobian matrix becomes poorly conditioned, meaning that small end-effector motions may require large joint velocities. This behavior appears in the plots as temporary increases in tracking error.



a) **b)**

Figure 5: End-effector tracking error X_{err} for the new-configuration case controller. a) angular error components as a function of time step and b) linear error components as a function of time step.

The manipulability plot in Figure 6 helps illustrate how the robot's configuration changes during the task. The angular manipulability factor $\mu_1(A_w)$ increases rapidly early in the motion and then stabilizes around a constant value for most of the trajectory. The linear manipulability factor $\mu_1(A_v)$ varies more throughout the motion, reflecting the different arm configurations required during the approach, transport, and placement phases of the task.

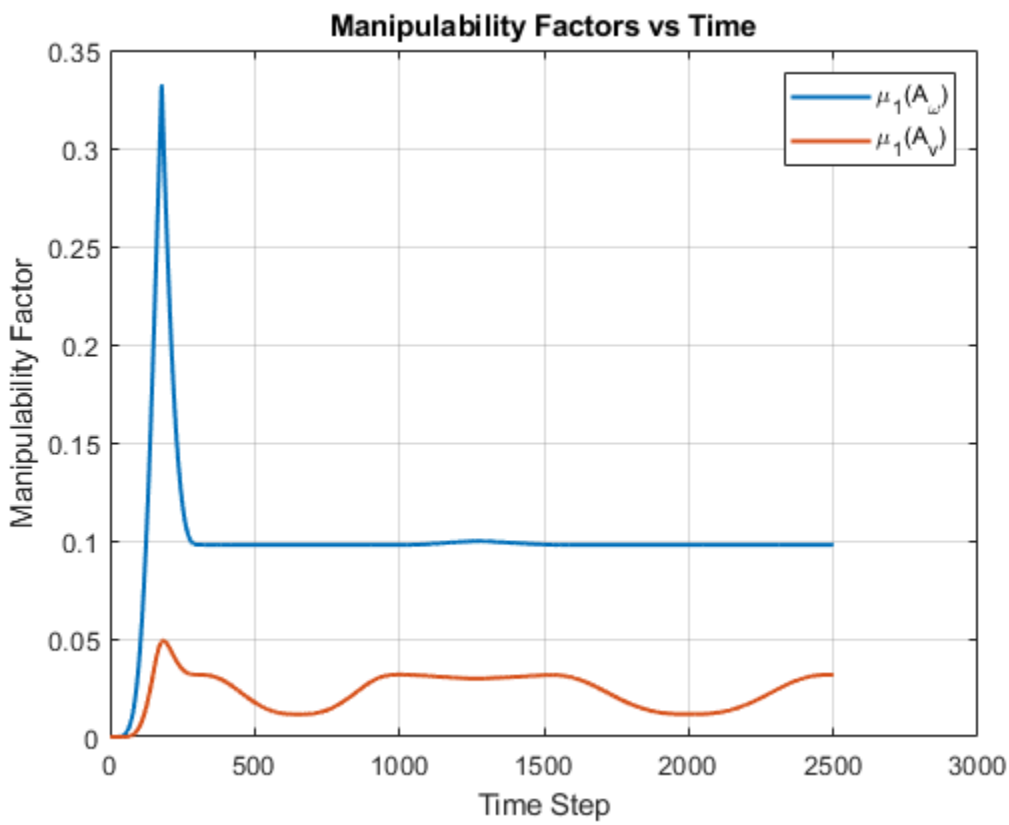


Figure 6: Manipulability factors $\mu_1(A_w)$ and $\mu_1(A_v)$ versus time for the new-configuration case controller. The angular and linear manipulability measures are plotted as the robot executes the pick-and-place trajectory.

Although the robot passes through configurations where the manipulability temporarily decreases, the damped least-squares method maintains numerical stability and prevents excessively large control commands. As a result, the controller quickly corrects the error and the robot successfully completes the new pick-and-place task without allowing the overshoot to accumulate large tracking errors.

Discussion

Increasing the integral gain K_i helps eliminate steady-state error because the controller accumulates past error and continues correcting the system until the tracking error approaches zero. However, if K_i is too large, the accumulated error causes the controller to apply excessive

corrective action, even after the system begins approaching the desired trajectory, This leads to overshoot and oscillations, as observed in Figure 3. Therefore, while the integral term improves long-term accuracy, excessively large values of K_i can make the system less stable and produce large transient responses whenever the robot is near a singularity.

The manipulability factor is a scalar measure that indicates how well the robot can generate motion of the end effector from the joint motions at a given configuration. It is derived from the Jacobian matrix of the robot and reflects how well-conditioned the Jacobian is. As seen in Figure 6, the manipulability factors become largest during step 1 of the trajectory when the end-effector moves from its initial configuration to the standoff configuration above the cube. At the beginning of the motion, the robot starts close to its home configuration where several joint angles are aligned, which places the manipulator near a singularity. Therefore, as the robot moves away from this point the Jacobian is better conditioned, allowing the robot to generate motion more effectively in multiple directions. Later in the trajectory, as the robot approaches the grasp and placement configurations, the manipulability decreases again because the arm joints move into configurations that are closer to singularities because the links of the arm are almost collinear.

When the maximum joint velocities are reduced, the robot cannot move its joints fast enough to follow the desired end-effector trajectory. After the cube is picked up, the robot must transport the object while following the planned trajectory generated by the trajectory generator function. If the velocity limits are too small, the commanded velocities from the controller are limited by the maximum joint velocity constraint. As a result, the robot cannot keep up with the reference motion and begins to lag behind the desired trajectory. This causes the tracking error to increase again even though the controller is trying to correct it.

Velocity control is realistic for tasks where the robot must smoothly follow a planned trajectory, such as moving the end-effector to pick up an object and transport it to another location which is done in all of the 3 cases above. In these cases the controller can command joint velocities to track the desired path continuously, which is appropriate for motions like the pick-and-place task done above. However, velocity control would not be suitable for tasks that require precise force interaction with the workspace, such as inserting a peg into a hole or tightening a bolt. In these cases the robot must regulate contact forces and torques, and simply using joint velocities would not provide enough control to safely perform the task.

Changing from velocity control to torque control, FeedbackControl would need the robot's dynamic model, not just its kinematics. In the Modern Robotics code, the core function for computing the required joint torques is the InverseDynamics function. Therefore, in addition to the current joint positions and velocities,the function needs the desired joint accelerations (ddthetalist), the gravity vector (g), end-effector wrench (Ftip), robot's home link transforms (Mlist), spatial inertia matrices (Glist) for each link. These are the quantities the MR library uses to map a desired motion into required torques. However, for full torque control of the entire youBot, chassis and wheels dynamic parameters and a model of how wheel torques interact with the ground is required in order to get full torque control of the robot.

Yes, generative AI was used as a support tool during the completion of this project. ChatGPT was used to help review explanations of robotics concepts, clarify portions of the Modern Robotics algorithms, and assist with improving the clarity and structure of the written report. The AI tool was also used to help edit and refine the written responses for the report sections.